MediSaarthi — Technical Proposal

Audience: Novalantis engineering (Yash — AI/orchestration, Shivam — ops/clinical workflows, Anupam — product/GTM). Scope: the *how and why* behind `architecture.html` / `architecture.pdf` and `BUILD_PLAN.md`. Read those first for the what/when; this document is the design reasoning underneath the boxes.

---

1. Design principles

Four decisions shape everything below, made explicit now so later choices can be checked against them instead of re-litigated:

1. **Safety-critical decisions are deterministic, not LLM judgment.** The red-flag guardrail is a rule match, not "ask the model if this sounds serious." An LLM's job in this system is to converse and retrieve; a symptom list and a transfer trigger are not something we let the model decide on its own, because a missed escalation is the one failure mode that actually hurts someone.
2. **Reuse the tenant boundary, don't reinvent it.** `auth.py`'s `company_id` scoping is the one piece of Talkly that's already production-hardened for multi-tenant isolation. Every new collection and service in this proposal is designed to sit inside that boundary unchanged, not to grow a parallel isolation mechanism.
3. **Grounding beats prompting.** "Never diagnoses" is not enforced by a system-prompt instruction — prompts drift and can be argued with. It's enforced by what content the agent physically has access to: if the discharge instructions aren't in its context, it can't repeat them wrong.
4. **Design for what Bolna actually exposes, not what the pitch deck assumes.** Bolna's documented mechanism is `agent_id` + recipient + webhook + prompt-template variables (confirmed in `technical_document.md` §8) — there is no confirmed live mid-call API in this codebase today. Every design below is written to work with that mechanism first, with the more ambitious version flagged as conditional on confirming Bolna's actual tool-calling support.

---

2. Call-handling: inbound and outbound

Inbound (reused, unchanged): `inbound/` already does exactly what MediSaarthi needs for a patient-initiated call — Twilio receives the call, `inbound/controller.py` / `routes.py` resolve the hospital from the dialed number, and TwiML `<Dial><Sip>` hands off to Bolna. `language_service.resolve_bolna_agent_id()` already resolves the right agent per `company_id` + language. Nothing here needs to change for MediSaarthi; a hospital's inbound line is configured exactly like a company's inbound line today.

Outbound (reused pattern, new trigger source): the bulk call-trigger logic inline in `server.py` already does "look up company's agent, call Bolna's API with recipient + prompt variables." For reminders/adherence, the only new piece is *what decides to call and when* — a `scheduler_logic` component that reads `appointments` / `consent_records` for due reminders and due adherence calls, and invokes the same trigger path. This is a cron-style job, not a rewrite of the trigger mechanism itself.

---

3. The red-flag guardrail — the part that actually matters

This is the one component with no existing analog in Talkly (`transfer_service.py` is a post-call heuristic over `analysis_result`, not a live hook — see `BUILD_PLAN.md` §2.2), so it gets the most design attention here.

3.1 What triggers it

Each hospital supplies a `protocols` document of type `redflag_list`: a structured list of symptom phrases / patterns per language, written by clinical staff at the pilot site, not by us. The guardrail is a **match against this hospital-supplied list**, evaluated against the running transcript on every turn. We are not building a general-purpose symptom classifier — we are building a lookup against a list a clinician wrote, which is a much easier and much more auditable thing to get right for a POC.

Match strategy, in order of preference:

1. **Phrase/keyword match** (language-aware, using the same Devanagari/script-range detection already in `language_service.detect_language_from_text`) — deterministic, explainable, trivially auditable in a review.
2. **Fallback: embedding similarity** against the hospital's red-flag phrase list, only if keyword match proves too brittle in pilot testing (e.g., misses paraphrases). This is a second phase, not POC day one — don't build it until keyword match is shown to be insufficient on real pilot transcripts.

Explicitly **not** doing: asking an LLM "does this sound like an emergency?" as the trigger itself. An LLM can be *part* of producing the transcript turn that gets checked, but the check is the deterministic list match. This keeps the guardrail's behavior reviewable by a clinical advisor as a static list, not as opaque model behavior.

3.2 What happens on a match

Two things need to happen inside the same conversation turn:

1. The call must stop the scripted flow and reach a human — this is the part that depends on Bolna's actual capability (§1, principle 4). The two viable mechanisms, in order of preference, pending confirmation:
   - **Bolna agent-side tool-calling**, if supported: the agent's tool definitions include a `transfer_call` function; the red-flag match is surfaced to the agent (e.g., as a mid-call event or a pre-registered condition Bolna itself evaluates) and Bolna executes the SIP transfer. This is the only approach that doesn't require us to build new real-time infrastructure.
   - **Fallback**: if Bolna has no mid-call tool-calling, the guardrail match has to come from a live transcript tap (if Bolna streams partial transcripts) that we evaluate and act on via Bolna's call-control API. This is materially more engineering — a reason to resolve this unknown in week one of Phase 0, not assume it away.
2. A `redflag_events` document is written (`call_id`, `patient_id`, `matched_phrase`, `matched_list_version`, `transferred_at`) **synchronously, before or at the same moment as the transfer**, not as a post-call cleanup step — if the transfer succeeds but the write fails, we lose the one piece of evidence that proves the safety mechanism worked, which matters both operationally and for the grant close-out metric (`BUILD_PLAN.md` §4).

3.3 Context delivered to the supervisor

Reuse the shape of `transfer_service.generate_transfer_context()` — it already returns a sensible payload shape (call_id, customer identity, summary, transcript snippet, language) — but the trigger condition changes from sentiment/intent heuristics to the red-flag match, and the delivery needs to be live (pushed to the Supervisor Dashboard, not read on the next page load). The dashboard's existing polling/analytics pattern needs a lightweight live-queue endpoint added; a full websocket layer is more than this needs for a 1–2 site POC — short-interval polling (a few seconds) from the dashboard is an acceptable POC-grade substitute for a push channel.

---

4. Knowledge grounding (RAG) — scoped for what Bolna can actually consume

Bolna's call-trigger mechanism takes `agent_welcome_message` and prompt-template variables, not an arbitrary live retrieval call mid-conversation (per `technical_document.md` §8, Talkly "only sends… prompt-template variables"). That constrains the realistic POC design:

- **Grounding at call-trigger time, not mid-call retrieval.** When a call is triggered — inbound (once the patient states department/reason) or outbound (department already known from the appointment) — the backend retrieves the relevant slice of that hospital's `protocols` documents (department directory entry, relevant discharge template, relevant schedule) and injects it as a prompt-template variable, the same mechanism already used for `user_data` variables today.
- **Chunking**: store `protocols` documents pre-split by department/topic at upload time (hospital admin uploads a schedule/protocol doc through a simple structured form or file, not a free-text blob we have to chunk automatically for POC). This avoids building a general document-chunking/embedding pipeline before we know we need one — a hospital's protocol set for a POC pilot is small enough to manage as structured records, not a full RAG index.
- **Mid-call updates** (e.g., if the patient's stated reason changes which protocol applies) are a stretch goal, not a POC requirement — flag it as explicitly out of scope rather than silently under-building it.

This is a deliberately smaller claim than "RAG" implies in the pitch deck — it's closer to **context injection from a small, structured, per-hospital knowledge base** than a general retrieval pipeline. That's the right scope for a 6-month POC with 1–2 sites; a real embedding/retrieval system is justified once there's enough protocol volume across more hospitals to need it.

---

5. Data model notes (concurrency, integrity)

- **`appointments` double-booking**: slot booking must be an atomic `find_one_and_update` with a status/availability filter (`{"slot_time": ..., "status": "open"}` → set `"booked"`), not a read-then-write — two concurrent inbound calls (or an inbound call racing a reminder-triggered reschedule) must not both succeed against the same slot. This is the same class of bug the existing `wallets` atomic-increment pattern in `billing_service.py` already avoids correctly — follow that precedent.
- **`consent_records`** : a call must not proceed past the point of collecting/using PII without a `granted_at` entry for that patient; check this at the point the call is triggered (outbound) or at first PII-bearing question (inbound), not as a post-hoc compliance check.
- **`audit_logs`** : insert-only. No route exposes update or delete on this collection — app-level enforcement is sufficient for a POC (don't build hash-chaining or a separate immutable store before a regulator or auditor asks for it). Index on `conversation_id` and `timestamp` for retrieval during the grant close-out audit.
- **`redflag_events`** : doubles as both a safety log and the grant's primary evidence artifact (`BUILD_PLAN.md` §4's "zero missed escalations" metric) — treat its schema as reviewer-facing, not just internal debugging data.

---

6. Integration design

- **HIS/CRM**: `crm_service.py` today is a mock stub (`asyncio.sleep(1)` , sets a flag) — there is no real adapter to extend. Rather than build a "generic HIS connector" that tries to anticipate every hospital's system, define a small adapter interface (`push_appointment` , `pull_schedule` , `push_outcome`) and implement one concrete adapter per pilot site. Treat the interface as a seam for future reuse, not as a

deliverable that has to be hospital-agnostic on day one — that generality isn't buyable with a 2-site POC's worth of evidence anyway.

- **ABDM**: scope strictly to the "building blocks" the deck itself names — facility-registry alignment and consent-artifact shape — not health-record exchange. Confirm sandbox/API access timeline before Phase 3 planning locks in, since this is the one dependency entirely outside our control.

---

7. Open technical decisions (resolve before Phase 1 is fully scoped)

1. Does Bolna support agent-side mid-call tool-calling/transfer? Gates §3.2's design — this is the single highest- leverage question to answer first.
2. Does Bolna expose a live/partial transcript stream, as a fallback if (1) is no? Determines whether §3.2's fallback is buildable at all within the grant's engineering budget.
3. What's the realistic size/format of a pilot hospital's actual protocol documentation? Determines whether the structured-record approach in §4 holds or whether we need lightweight chunking sooner than planned.